Date .....

| Notes |

Vegetable_Dataset

↓

scan folders (Parsh (fresh/rotten) + vetebable names)

↓

Filters only target vegetables → Tomato, carrot, Potato, Cucumber, capsicum

↓

Load Images

↓

Resize (224x224) pixel

↓

Feature Extraction

→ EfficientNetB0
1280 deep features

→ Handcrafted features
- → Edge density
- → Laplacian variance
- → Histogram bins
- → grayscale stats
- → RGB mean/std
- → HSV mean/std

32 features

→ 1312 feature vector per image

↓

Feature matrix

X.npy (N x 1312)

Y_veg.npy

Y_fresh.npy

|Notes|

Date .....

X.npy
Y_veg.npy
Y_fresh.npy

↓

Train_split.py

↓

Stratified Split
(vegetable + Freshness)

→ X_train.npy
Y_veg_train.npy
Y_fresh_train.npy

→ X_test.npy
Y_veg_~~trai~~test.npy
Y_fresh_test.npy

X_train.npy

↓

preprocess_and_rank.py

↓

Variance Threshold (remove constant features)

↓

StandardScalar (Feature Normalization)

↓

XGBoost Feature Ranking
(label = vegetable + Freshness)

↓

Select top 100 features

↓

Save preprocessing artifact

→ Variance.joblib
scalar.joblib
Selected_features.~~joblib~~ npy
feature_importance.npy

| Notes |

Date .....

Before model training the exact same preprocessing pipeline must be applied to X_train using the saved artifact.

1. load (variance.joblib)
2. X_train
   ↓
   variance.transform (X_train) removes constant features.
   ↓
   X_train_vt (N×1312) → (N× ~1200)

1. load (scalar.joblib)
2. X_train_vt
   ↓
   scalar.transform () mean=0, std=1 normalize feature distribution
   ↓
   X_trained_scaled

1. load (selected_features.npy)
2. X_train_scaled
   ↓
   X_train_scaled [:, selected_features]
   ↓
   X_train_final (N×100)

| Notes |

Date .....

XGBoost calculates how important each features is for prediction. (vegetable + Freshness)

result [0.021, 0.004, 0.033, 0.0001, 0.014, ...]

length ≡ no.of features after varience threashold.

This array is saved as

feature_importance.npy → used for visualization only.

↓

Pick top 100 indices

↓

selected_features.npy

↓

used for training + prediction

Train vegetable classifier

1. load (y_veg_train.npy)

2. ↓ label Encoading

Tomato → 0 Capsicum → 3

carrot → 1 cucumber → 4

Potato → 2

saved as

(label_encoder.joblib) y_veg_train_encoded

X_train_final + y_veg_train_encoded } → SVM1 → (veg_svm.joblib)

y_fresh_.npy → 0 ≡ Fresh
→ 1 ≡ Rotten

Date .....

| Notes |

Train Freshness classifier

1. load (y_fresh_train.npy)
2. labelEncoding → no need

fresh → 0
rotten → 1

no_joblib_file is created

X_train_final
y_fresh_train_~~encoded~~ } SVM2 → (fresh_svm.joblib)

→ now we have,

variance.joblib	X_train.npy
scalar.joblib	X_test.npy
selected_features.npy	X_veg_train.npy
feature_importance.npy	X_veg_test.npy
label_encoder.joblib	Y_fresh_train.npy
veg_svm.joblib	Y_fresh_test.npy
fresh_svm.joblib	

|Notes|

Date .....

Model Evaluation (src| evaluate_models.py)

X_test.npy

↓

variance.transform()

↓

scalar.transform()

↓

select_features

↓

X_test_final

#Vegetable Prediction and Freshness Prediction

Veg_svm.predict(X_test_final) ⟶ Predicted vegetable

fresh_svm.predict(X_test_final) ⟶ Fresh|Rotten

Evaluation Metrices

Accuracy
Classification Report
Confusion Matrix
] → For both model

| Notes |

Date .....

Prediction Pipeline (Real Image) Predict_cli.py

Input Image
↓
Resize (224x224)
↓
Feature Extraction (1312 Features)
↓
variance.transform — variance.joblib
↓
scalar.transform — scalar.joblib
↓
selected_features — selected_features.npy
↓
veg_svm.predict — veg_svm.joblib
↓
Fresh_svm.predict — fresh_svm.joblib

Freshness Score + Freshness Grade

Freshness Probability (P(Fresh))
↓
score = P(Fresh) x100

Ex P(Fresh) = 0.82
Score = 82

score ≥ 95 → Fully Fresh
score ≥ 75 → mostly Fresh
score ≥ 50 → Medium
score < 50 → Rotten